

Strided Implementation of Hypothesis Tests for `stdlib`

GSoC 2026 Proposal
Submitted by: Sumithra Raju

Title

****Strided Array Support for Statistical Hypothesis Tests in `stdlib`****

Abstract

This proposal aims to extend stdlib's statistical hypothesis testing functionality by implementing strided array support across 14 major hypothesis tests. Currently, these tests accept only contiguous arrays, limiting performance optimization and usability with non-contiguous data. By implementing strided variants (dttest, sttest, etc.) for both JavaScript and C, we will significantly improve performance for applications working with sliced, transposed, or multi-dimensional array views. This work prioritizes JavaScript implementation first, followed by C implementations with careful attention to blocking dependencies (t.cdf, t.quantile requiring betainc kernel family functions). The project focuses on ttest and ttest2 as primary targets, establishing patterns for the remaining 12 tests.

Technical Details

Project Scope

****Objective****: Implement strided array support for 14 statistical hypothesis tests in stdlib, enabling efficient computation on non-contiguous data views while maintaining numerical accuracy and API compatibility.

****14 Tests Requiring Strided Implementation****:

1. ttest (one-sample t-test) - ****PRIORITY 1****
2. ttest2 (two-sample t-test) - ****PRIORITY 1****
3. pccortest (Pearson correlation test)
4. chi2gof (Chi-square goodness-of-fit)
5. bartlettTest (equal variances)
6. kruskalTest (Kruskal-Wallis)
7. leveneTest (equal variances)
8. flignerTest (equal variances)
9. kstest (Kolmogorov-Smirnov)
10. vartest (F-test for variances)
11. wilcoxon (Wilcoxon signed rank)
12. binomialTest (binomial test)
13. anova1 (one-way ANOVA)
14. chi2test (independence test)

Architecture Overview

Directory Structure

stats/strided/

■■■■ dttest/ # double-precision float (Float64)

■ ■■■ lib/

■ ■■■ index.js

■ ■■■ ndarray.js

■ ■■■ src/

■ ■■■ main.c

■■■ sttest/ # single-precision float (Float32)

■ ■■■ lib/

■ ■■■ index.js

■ ■■■ ndarray.js

■ ■■■ src/

■ ■■■ main.c

■■■ ttest/ # generic interface

■■■ lib/

■■■ index.js

■■■ ...

stats/base/ttest/

■■■ one-sample/

■ ■■■ results/

■ ■■■ factory/

■ ■■■ float64/

■ ■■■ float32/

■ ■■■ struct-factory/

■ ■■■ to-json/

■ ■■■ to-string/

■■■ (two-sample addition for ttest2)

■■■ ...

Implementation Strategy

Phase 1: JavaScript Implementation

1. **tttest (one-sample)**

- Implement `stats/strided/dttest` (double-precision)
- Implement `stats/strided/sttest` (single-precision)
- Implement `stats/strided/ttest` (generic wrapper)
- Tests and documentation

2. **tttest2 (two-sample)**

- Add two-sample to `stats/base/ttest`
- Implement `stats/strided/dttest2` (double-precision)
- Implement `stats/strided/sttest2` (single-precision)
- Implement `stats/strided/ttest2` (generic wrapper)

Phase 2: C Implementation (with Dependency Analysis)

- **Blockers**: t.cdf and t.quantile require betainc kernel family
- Evaluate existing C implementations of special functions
- Implement direct C translations of validated JS logic
- Focus on linear-algebra optimizations for performance

Phase 3: Additional Tests (Pattern Replication)

- Apply ttest/ttest2 pattern to remaining 12 tests
- Each test follows same strided architecture
- Identify additional C blockers and dependencies

Phase 4: Float16 Support

- Implement Float16 variants after Float32/Float64 validation
- Ensure precision handling for reduced-precision computation

Phase 5: Complex32 Support

- Extend to complex number support where applicable
- Limited to tests supporting complex data (correlations, certain distributions)

Key Technical Considerations

Strided Array Access Pattern

// Generic strided access

```
for (let i = 0; i < n; i++) {  
  value = arr[offset + i * stride];  
}
```

Data Type Support Matrix

C Implementation Dependencies

- **t.cdf**: Requires betainc (incomplete beta function)
- Current status: Check if kernel-betainc exists
- Alternative: Implement using Lanczos approximation
- **t.quantile**: Inverse of t.cdf
- Dependency: t.cdf functionality
- Implementation: Newton-Raphson method or lookup table

Libraries and Technologies

- **Runtime**: Node.js with native modules (N-API)
- **Testing**: Tape (stdlib test framework)
- **Math Libraries**: stdlib's own special functions
- **Version Control**: Git with semantic versioning
- **Documentation**: JSDoc, inline comments, README files

Literature and References

- [Student's t-distribution CDF](https://en.wikipedia.org/wiki/Student%27s_t-distribution)
- [Incomplete Beta Function](https://en.wikipedia.org/wiki/Beta_function)
- Numerical Recipes: Chapter on Special Functions
- stdlib existing implementations: ``lib/node_modules/@stdlib/math/base/special/``

Schedule of Deliverables

Community Bonding Period (Week 1-2)

Objective: Establish project foundation and community alignment

- [] Read and understand existing ttest/ttest2 implementations
- [] Review stdlib coding standards and contribution guidelines
- [] Set up development environment and testing infrastructure
- [] Identify blocking dependencies (t.cdf, t.quantile availability)
- [] Create issue on stdlib GSoC tracker with research findings
- [] First blog post: "Exploring Strided Arrays in stdlib"

Deliverables:

- Setup documentation
- Dependency analysis report
- Community introduction post

Phase 1: JavaScript Implementation (Week 3-8)

Week 3-4: ttest (One-Sample) - Float64 Variant

Duration: 10 working days | **Effort**: 40 hours

Subtasks:

1. Analyze existing `stats/base/ttest` structure (2 hours)
2. Implement `stats/strided/dttest/ndarray.js` (6 hours)
3. Implement `stats/strided/dttest/lib/index.js` (4 hours)
4. Write comprehensive test suite (8 hours)
5. Documentation and JSDoc (4 hours)
6. Code review and adjustments (2 hours)

Acceptance Criteria:

- [] All tests pass (100% coverage for strided paths)
- [] Performance benchmarks show $\geq 10\%$ improvement for strided data
- [] Documentation complete with usage examples

Week 5: ttest (One-Sample) - Float32 Variant

Duration: 5 working days | **Effort**: 20 hours

Subtasks:

1. Implement `stats/strided/sttest/ndarray.js` (5 hours)
2. Implement `stats/strided/sttest/lib/index.js` (3 hours)
3. Write test suite (8 hours)
4. Documentation (2 hours)
5. Cross-precision validation (2 hours)

Week 6: ttest (One-Sample) - Generic Wrapper

****Duration****: 3 working days | ****Effort****: 12 hours

****Subtasks****:

1. Implement `stats/strided/ttest/` generic interface (4 hours)
2. Type dispatch logic (4 hours)
3. Integration tests (3 hours)
4. Documentation (1 hour)

****Deliverable****: Fully functional strided ttest implementation

Week 7-8: ttest2 (Two-Sample) - JavaScript Implementation

****Duration****: 10 working days | ****Effort****: 40 hours

****Subtasks****:

1. Add two-sample variant to `stats/base/ttest/` (6 hours)
2. Implement `stats/strided/dttest2/ndarray.js` (8 hours)
3. Implement `stats/strided/sttest2/ndarray.js` (6 hours)
4. Implement `stats/strided/ttest2/` generic wrapper (4 hours)
5. Comprehensive test suite (10 hours)
6. Documentation (4 hours)
7. Code review and optimization (2 hours)

****Phase 1 Deliverable****:

- ✓ Strided implementations of ttest and ttest2
- ✓ Full test coverage for both tests
- ✓ Performance benchmarks
- Blog post: "Implementing Strided t-tests in JavaScript"

Phase 2: C Implementation & Dependency Resolution (Week 9-12)

Week 9: Dependency Analysis and Planning

****Duration****: 5 working days | ****Effort****: 20 hours

****Subtasks****:

1. Audit existing special function implementations (6 hours)
2. Check for t.cdf and t.quantile availability (4 hours)
3. Plan betainc kernel implementation (6 hours)
4. Design C module architecture (4 hours)

****Deliverable****: Dependency resolution plan with recommendations

Week 10-11: C Implementation for ttest/ttest2

****Duration****: 10 working days | ****Effort****: 40 hours

****Subtasks**** (if dependencies resolved):

1. Implement C bindings for dttest (8 hours)
2. Implement C bindings for sttest (6 hours)

3. Implement C bindings for dttest2 (8 hours)
4. Implement C bindings for sttest2 (6 hours)
5. Performance testing and optimization (8 hours)
6. Documentation (4 hours)

****If blocker encountered****:

- Escalate to mentors with technical analysis
- Continue with Phase 3 pattern implementation
- Plan C implementation for Phase 2 continuation

Week 12: Integration and Testing

****Duration****: 5 working days | ****Effort****: 20 hours

****Subtasks****:

1. Integration tests between JS and C (8 hours)
2. Cross-platform testing (4 hours)
3. Performance benchmarking (4 hours)
4. Documentation update (4 hours)

****Phase 2 Deliverable****:

- ✓ C implementations (if dependencies available)
- ✓ Integration tests
- ✓ Performance benchmarks vs JS
- Blog post: "C Acceleration for Statistical Tests"

Phase 3: Pattern Replication (Week 13-15)

Week 13: Remaining Tests - Part 1

****Duration****: 5 working days | ****Effort****: 20 hours

Implement strided variants for:

1. pccortest (Pearson correlation)
2. chi2gof (Chi-square goodness-of-fit)
3. bartlettTest (Bartlett's test)

****Approach****:

- Apply ttest pattern to each test
- Minimal analysis of each test's computation
- Focus on striding logic, not algorithmic changes

Week 14: Remaining Tests - Part 2

****Duration****: 5 working days | ****Effort****: 20 hours

Implement strided variants for:

1. kruskalTest (Kruskal-Wallis)
2. leveneTest (Levene's test)
3. flignerTest (Fligner-Killeen test)

Week 15: Remaining Tests - Part 3 + Integration

****Duration****: 5 working days | ****Effort****: 20 hours

Implement strided variants for:

1. kstest (Kolmogorov-Smirnov)
2. vartest (Variance test)
3. wilcoxon (Wilcoxon signed-rank)
4. binomialTest
5. anova1
6. chi2test

Plus integration and cross-test validation.

****Phase 3 Deliverable****:

- ✓ 12 additional tests with strided support
- ✓ Pattern documentation for maintainers
- ✓ Full test coverage
- Blog post: "Scaling Hypothesis Tests with Strided Arrays"

Phase 4: Float16 Support (Optional, Post-GSoC)

****Subtasks****:

1. Float16 variant implementation for all tests (15 hours/test)
2. Precision handling and accuracy validation
3. Cross-precision testing
4. Documentation updates

****Estimated Duration****: 3-4 weeks (after core implementation)

Phase 5: Complex32 Support (Optional, Post-GSoC)

****Subtasks****:

1. Identify applicable tests (correlations, distributions with complex parameters)
2. Complex number arithmetic implementation
3. Algorithm validation for complex domain
4. Testing and documentation

****Estimated Duration****: 2-3 weeks (after Float16)

Final Week (Week 16): Code Quality & Submission

****Deliverables****:

- [] All code submitted to stdlib repository
- [] Pull requests merged or approved
- [] Final blog post: "GSoC Project Summary"

- [] Final documentation review
- [] Code coverage $\geq 95\%$
- [] Performance benchmarks published
- [] Contribution guide for future maintainers

Implementation Roadmap (Visual Timeline)

WEEK 1-2: [Community Bonding & Setup]

WEEK 3-6: [ttest JS Float64/Float32 + Wrapper]

WEEK 7-8: [ttest2 JS Implementation]

WEEK 9-12: [C Implementation & Dependencies]

WEEK 13-15: [12 Additional Tests]

WEEK 16: [Final Integration & Submission]

Development Environment & Tools

Build and Test Infrastructure

Environment setup

npm install

npm run build

Run tests

npm test

npm run test:coverage

Benchmarking

npm run benchmark

Documentation generation

npm run docs

Code Quality Standards

- **Linting**: ESLint (stdlib config)
- **Code Style**: stdlib JavaScript conventions
- **Testing Framework**: Tape
- **Documentation**: JSDoc + markdown
- **Coverage Target**: ≥95%

Collaboration Tools

- **Version Control**: GitHub (stdlib fork/upstream)
- **Issue Tracking**: stdlib GitHub issues + GSoC tracker
- **Communication**: GitHub discussions, weekly sync with mentors
- **Blog**: Personal blog for progress updates

Risk Assessment & Mitigation

Technical Risks

Timeline Risks

- **Mitigation**: Build 2-3 week buffer in Phase 3 for unexpected issues
- **Fallback**: Prioritize ttest/ttest2; remaining tests are less critical

Success Criteria

Primary Deliverables (Must-Have)

- [x] Full strided implementation of ttest and ttest2 (JS + C if possible)
- [x] 95%+ test coverage for implemented functions
- [x] Performance benchmarks showing $\geq 10\%$ improvement for strided data
- [x] Complete documentation and usage examples
- [x] All code merged to stdlib main branch

Secondary Deliverables (Should-Have)

- [] 12 additional tests with strided support (Phase 3)
- [] C implementations of all tests
- [] Performance benchmarks for C vs JS
- [] Contribution guide for future implementations

Nice-to-Have Deliverables

- [] Float16 support
- [] Complex32 support
- [] SIMD optimizations
- [] Comprehensive benchmark suite

Previous Work & Community Context

Existing Contributions

Based on the memory:

- Completed: 2 tests (characterized as "easy ones")
- Current: Strided implementation approach design
- Previous discussion: #179 on stdlib repository

Community Feedback (from Uday Kakade)

1. JS implementation must come before C
2. C blockers: t.cdf and t.quantile
3. Priority order: ttest $\rightarrow$ ttest2 $\rightarrow$ others
4. Check for C dependencies before committing to C impl

Mentor Guidance (from Athan Reines)

1. Start with easier tests first
2. Prioritize JS over C (identify blockers early)
3. Identify prerequisites before starting
4. Maintain regular communication with community

Personal Background & Motivation

Development Experience

- **Experience Level**: Full-stack JavaScript/C development
- **Previous Contributions**: stdlib statistical tests
- **Relevant Coursework**: Statistics, numerical methods, algorithms
- **Open Source**: Active stdlib contributor

Why This Project?

1. **Technical Depth**: Bridges JavaScript and C implementation
2. **Real-World Impact**: Improves performance for scientific computing
3. **Community**: Opportunity to work with active stdlib community
4. **Learning**: Deep dive into statistical algorithms and optimization

Goals for GSoC

1. Deliver production-ready strided hypothesis test implementations
2. Establish pattern for future strided array implementations
3. Contribute significantly to stdlib's statistical functionality
4. Learn advanced optimization techniques for numerical computing

Appendix

A. Test Case Template

```
// stats/strided/dttest/test/test.ndarray.js
const test = require('tape');
const dttest = require('../../lib/ndarray');
test('dttest( x, stride, offset, options )', (t) => {
  t.test('returns object with correct properties', (t) => {
    // Test implementation
  });
  t.test('computes correct test statistic', (t) => {
    // Validate against reference implementation
  });
});
```

B. Performance Benchmark Template

```
// stats/strided/dttest/benchmark/benchmark.ndarray.js
const bench = require('@stdlib/bench');
const dttest = require('../../lib/ndarray');
```

```
const suite = new Bench.Suite();  
suite  
.add('dttest: contiguous array', () => {  
  // Benchmark  
})  
.add('dttest: strided array', () => {  
  // Benchmark  
})  
.run();
```

C. Documentation Template

```
# strided ttest  
> One-sample t-test with strided array support
```

Usage

Arrays

```
const { dttest } = require('@stdlib/stats/strided/ttest');  
const x = new Float64Array([...]);  
const result = dttest(x, 1, 0, { mu: 0 });
```

Strides

```
const stride = 2; // skip every other element  
const result = dttest(x, stride, 0, { mu: 0 });
```

D. Dependency Checking Script

```
#!/bin/bash  
# Check for required C dependencies  
echo "Checking for t.cdf availability..."  
find /lib/node_modules/@stdlib -name "**t*cdf*" -type d  
echo "Checking for t.quantile availability..."  
find /lib/node_modules/@stdlib -name "**t*quantile*" -type d  
echo "Checking for betainc availability..."  
find /lib/node_modules/@stdlib -name "**betainc*" -type d
```

E. GitHub Issue Template

Strided Implementation: [Test Name]

Description

Implement strided array support for [test name]

Implementation Plan

1. JavaScript (Float64)
2. JavaScript (Float32)
3. Generic wrapper
4. C implementation (if dependencies available)

Blockers

- [] Dependencies identified
- [] API design approved

Files to Create

- `stats/strided/d[test]/`
- `stats/strided/s[test]/`
- `stats/strided/[test]/`

Testing

- Unit tests for all strided paths
- Performance benchmarks
- Cross-precision validation

F. Blog Post Templates

Post 1: Community Bonding

Getting Started with stdlib's Statistical Tests

This post covers:

- Project overview
- stdlib's test structure
- Development environment setup
- Initial dependency research

Post 2: Implementation Deep Dive

Implementing Strided t-tests in JavaScript

This post covers:

- Strided array pattern
- Algorithm implementation
- Performance characteristics
- Testing strategy

Post 3: C Acceleration

Optimizing Statistical Tests with C

This post covers:

- C binding architecture
- Dependency resolution
- Performance improvements
- Cross-platform testing

Post 4: Summary

GSoC 2026: Strided Hypothesis Tests Summary

This post covers:

- Project completion status
- Lessons learned
- Performance results
- Future work

References & Resources

Stdlib Documentation

- [stdlib stats namespace](https://github.com/stdlib-js/stdlib/tree/develop/lib/node_modules/@stdlib/stats)
- [Coding standards](<https://github.com/stdlib-js/stdlib/blob/develop/CONTRIBUTING.md>)
- [C binding guide](https://github.com/stdlib-js/stdlib/blob/develop/C_BINDINGS.md)

Mathematical References

- Student's t-distribution: [Wikipedia](https://en.wikipedia.org/wiki/Student%27s_t-distribution)
- Hypothesis testing: [Introduction to Statistics](<https://www.khanacademy.org/>)
- Numerical methods: Numerical Recipes in C

Tools & Libraries

- [Node.js N-API Documentation](https://nodejs.org/api/n_api.html)
- [Tape Testing Framework](<https://github.com/substack/tape>)
- [ESLint](<https://eslint.org/>)

Related Issues & Discussions

- [stdlib #179: Strided arrays discussion](<https://github.com/stdlib-js/stdlib/issues/179>)
- [GSoC Ideas: Statistical tests](<https://github.com/stdlib-js/stdlib/wiki/GSoC-Ideas>)

Sign-Off

****Prepared by****: Sumithra Raju

****Project****: Strided Implementation of Hypothesis Tests for stdlib

****GSoC 2026****

****Status****: Ready for review and mentor feedback

****Next Steps****:

1. ☐ Submit to stdlib GSoC issue tracker
2. ☐ Schedule kickoff meeting with mentors
3. ☐ Begin Phase 1 implementation
4. ☐ Publish first blog post